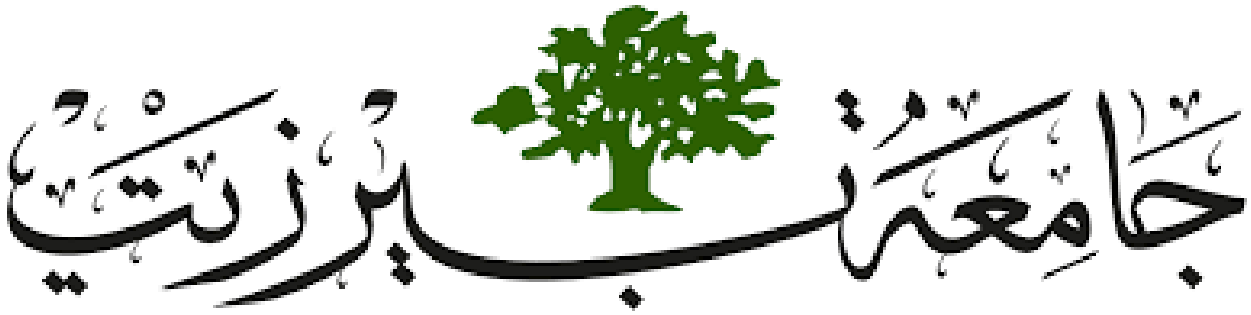


BIRZEIT UNIVERSITY

FACULTY OF ENGINEERING AND TECHNOLOGY

DEPARTMENT OF ELECTRICAL AND COMPUTER ENGINEERING

ENCS3310

Advanced Digital Design

Project Report

Prepared by:

Amer Qashoo

Supervised by:

Dr. Abdallatif Abuissa

Date:

6/9/2026

1 Abstract:

This project presents the design and verification of a special digital counter that generates one of two sequences: 1,4,7,… or 2,6,5,... The circuit is designed using JK flip-flops and combinational logic, with an asynchronous reset and a synchronous enable input. The selection input is latched before each counting cycle starts, and the stored value determines which sequence is used during that cycle. The design is implemented in Verilog and tested using a self-checking testbench to verify the counting sequence, reset, enable, and sequence switching.

Table of Contents

1 Abstract: I
2 Theory 1
2.1 Sequential Logic Circuits 1
2.2 JK Flip-Flop 1
2.3 JK Excitation Table 2
2.4 Asynchronous Reset 2
2.5 Synchronous Enable 2
2.6 State-Based Counter Design 2
2.7 Combinational Excitation Logic 3
2.8 Structural Verilog Design 3
3 Design Procedure 3
3.1 Design Assumptions and Decisions 3
3.1.1 Sequence Selection Convention 4
3.1.2 Initial and Unused States 4
3.1.3 Stored Selection Bit 4
3.1.4 Selection Capture Timing 4
3.1.5 Enable Implementation 4
3.2 State Encoding 4
3.3 Counter Sequences 5
3.3.1 Sequence A 5
3.3.2 Sequence B 5
3.3.3 Sequence Switching Behavior 5
3.4 State Diagram 6
3.5 State Transition Table 6
3.6 JK Excitation Table for the Counter 7
3.7 Karnaugh Maps 8
3.7.1 Karnaugh Maps for JK Excitation Inputs 8
3.8 Simplified Counter Equations 9
3.9 Overall Structural Architecture 9
3.10 Verilog Implementation 10
3.10.1 JK_FF Module 10
3.10.2 JK_FF_NEG Module 10
3.10.3 counter_logic Module 11

3.10.4 mode_logic Module 11
3.10.5 special_counter Top Module 11
4 Verification and Simulation Results 12
4.1 Testbench Methodology 12
4.2 Asynchronous Reset Test 12
4.3 Sequence A Test 12
4.4 Enable Hold Test 13
4.5 Selection Change While Disabled 13
4.6 Sequence B Test 13
4.7 Switching from Sequence B to Sequence A 13
4.8 Selection Change During an Active Cycle 13
4.9 Simulation Waveform 13
5 Discussion 14
5.1 Design Decisions 14
5.2 Advantages and Limitations 15
5.3 Suggested Improvements 15
6 Conclusion 16

Table Of Figures:

Figure 2.1: Basic structure of a sequential logic circuit. 1
Figure 2.2: JK flip-flop symbol 1
Figure 2.3: General example of state transitions in a counter. 3
Figure 2.4:General structural organization of the designed counter 3
Figure 3.1: State Diagram 6
Figure 3.2: K-maps 8
Figure 3.4:Overall structural block diagram of the special counter, as implemented in Active-HDL 9
Figure 3.5: JK_FF Module code 10
Figure 3.6 : JK_FF _NEG Module code 10
Figure 3.7: counter_logic Module code 11
Figure 3.8: mode_logic Module code 11
Figure 3.9:special_counter Top Module code 11
Figure 4.1:Verilog self-checking testbench used to verify the special counter 12
Figure 4.2: Initial asynchronous reset test 12
Figure 4.3: Asynchronous reset during operation test 12
Figure 4.4: Console output of the self-checking testbench showing that all test cases passed successfully. 14

List of Tables

Table 1: JK flip-flop operation table 1
Table 2: JK flip-flop excitation table 2
Table 3: State encoding of the special counter 5
Table 4: State transition table of the special counter 6
Table 5:JK flip-flop excitation table 7
Table 6:JK excitation table for the special counter 7

2 Theory

2.1 Sequential Logic Circuits

Sequential circuits are digital circuits whose outputs depend on both the current inputs and the previously stored state. In synchronous sequential circuits, the stored state is usually updated according to a clock signal. Flip-flops are commonly used as storage elements in this type of circuit.

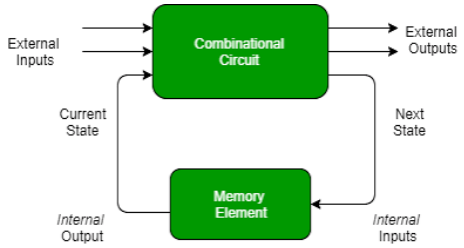


Figure 2.1: Basic structure of a sequential logic circuit.

2.2 JK Flip-Flop

The JK flip-flop is a memory element with two inputs, J and K, and one stored output Q. Depending on the values of J and K, the flip-flop can hold, reset, set, or toggle its current state. The course material describes the four JK operations as shown below.

Table 1: JK flip-flop operation table

J	K	Operation
0	0	Hold
0	1	Reset
1	0	Set
1	1	Toggle

Flip-flops are edge-triggered, meaning that their state changes at a specified transition of the clock. In this project, positive-edge-triggered JK flip-flops are used for the counter, while a negative-edge-triggered JK flip-flop is used for storing the selected sequence. Edge-triggered sequential circuits update their state only when the specified clock transition occurs.

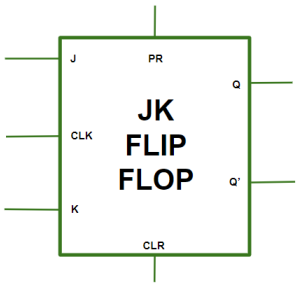


Figure 2.2: JK flip-flop symbol

2.3 JK Excitation Table

The excitation table is used to determine the required J and K values for a desired transition from the present state Q to the next state Q(next).

Table 2: JK flip-flop excitation table

Q	Q(next)	J	K
0	0	0	X
0	1	1	X
1	0	X	1
1	1	X	0

This table is used later to obtain the JK input values for each counter state.

2.4 Asynchronous Reset

An asynchronous reset changes the state of a flip-flop immediately when the reset signal becomes active, without waiting for a clock edge. This is implemented by including the reset signal in the sensitivity list alongside the clock edge, so a change in reset is detected immediately regardless of the clock.

In this project, the asynchronous reset returns the counter to its initial state.

2.5 Synchronous Enable

The enable input controls whether the counter is allowed to change its state. Since the enable is synchronous, its effect on the counter occurs only with the active clock edge.

When EN = 1, the counter follows the required state sequence. When EN = 0, the current state is held.

The project specification explicitly requires a synchronous enable to enable or disable counting.

2.6 State-Based Counter Design

A counter can be represented using a set of states, where each binary state represents a specific counter value. The required sequence is described by defining the transition from each present state to its next state.

The state transitions can then be represented using a state diagram and a state transition table. These are later used to determine the required JK flip-flop inputs.

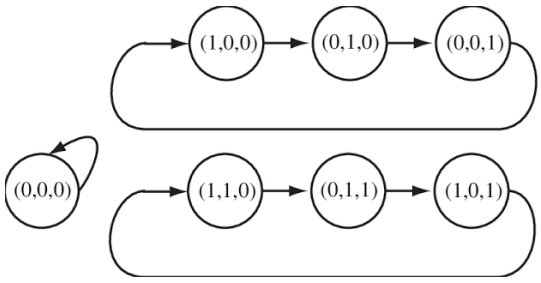


Figure 2.3: General example of state transitions in a counter.

2.7 Combinational Excitation Logic

The JK inputs are generated using combinational logic based on the present state and the control inputs. After determining the required J and K values from the excitation table, Karnaugh maps can be used to simplify the Boolean expressions.

The simplified equations are then connected to the JK flip-flops to produce the required next-state behavior.

2.8 Structural Verilog Design

Structural Verilog describes a circuit by connecting smaller modules together. In this project, the main counter is constructed using separate JK flip-flop modules and combinational logic modules. The combinational equations are described using continuous (assign) statements, while the top-level module connects the different parts of the circuit together. Continuous assignments are used in the course material to describe combinational hardware directly.

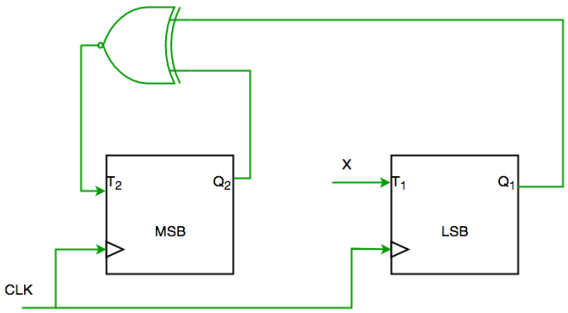


Figure 2.4:General structural organization of the designed counter

3 Design Procedure

3.1 Design Assumptions and Decisions

Several design decisions were made to implement the required counter in a clear and structured way.

3.1.1 Sequence Selection Convention

The input S selects the required counting sequence:

- S = 0 → 1 → 4 → 7
- S = 1 → 2 → 6 → 5

3.1.2 Initial and Unused States

State 000 is used as the initial state and is reached whenever the asynchronous reset is activated.

State 011 is not part of either required counting sequence and is treated as an unused state during logic simplification.

3.1.3 Stored Selection Bit

A separate bit M stores the selected sequence. The counter uses M instead of the external input S directly.

This prevents a change in S during an active cycle from immediately changing the current sequence.

3.1.4 Selection Capture Timing

The counter state flip-flops are triggered on the positive clock edge, while the flip-flop storing M is triggered on the negative edge.

This allows the new selection to be stored before the next positive edge starts the following sequence cycle.

3.1.5 Enable Implementation

The synchronous enable signal is included in the JK excitation logic.

- EN = 0 → the counter holds its current state.
- EN = 1 → normal counting continues.

This provides a simple implementation of the enable function.

3.2 State Encoding

Three flip-flops are used to represent the counter state, giving the state bits Q2Q1Q0. The binary representation of each required counter value is used directly as the state encoding.

Table 3: State encoding of the special counter

Function	Binary State	Decimal Value
Initial state	000	0
Sequence A	001	1
Sequence B	010	2
Unused state	011	3
Sequence A	100	4
Sequence B	101	5
Sequence B	110	6
Sequence A	111	7

3.3 Counter Sequences

The counter is designed to follow one of two different state sequences depending on the stored selection value M.

3.3.1 Sequence A

When the stored selection corresponds to Sequence A, the counter follows:

001 → 100 → 111 → 001 which represents: 1 → 4 → 7 → 1

3.3.2 Sequence B

When the stored selection corresponds to Sequence B, the counter follows:

010 → 110 → 101 → 010 which represents: 2 → 6 → 5 → 2

3.3.3 Sequence Switching Behavior

If the selection input changes during the current counting cycle, the current sequence is completed first. The new stored selection is then applied at the beginning of the next cycle.

For example, when the counter is following Sequence A and the selection changes to Sequence B, the counter completes: 001 → 100 → 111

and then starts the new sequence: 010 → 110 → 101

This behavior follows the requirement that the selection is latched before the beginning of a sequence cycle.

3.4 State Diagram

The state diagram shows the transitions between the counter states for both sequences. Sequence A uses the states 001, 100, and 111, while Sequence B uses 010, 110, and 101. The state 000 is used as the initial state after reset.

When EN = 0, the counter remains in its current state. When EN = 1, the counter moves to the next state according to the active sequence.

For the final state of each sequence, the next state depends on the stored selection value M:

- From state 111: if M = 0, the next state is 001; if M = 1, the next state is 010.
- From state 101: if M = 0, the next state is 001; if M = 1, the next state is 010.
- From state 000: if M = 0, the next state is 001; if M = 1, the next state is 010.

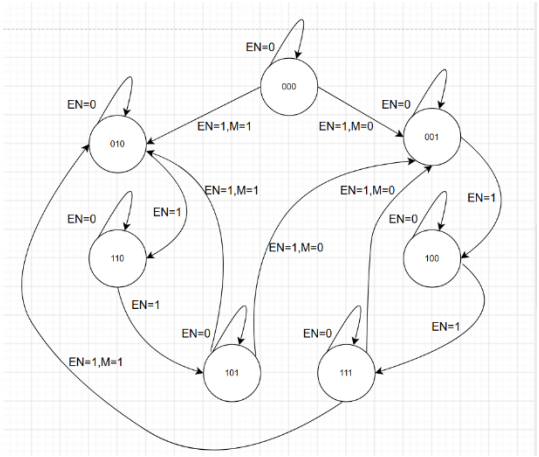


Figure 3.1: State Diagram

3.5 State Transition Table

The state transition table summarizes the next state of the counter for each valid present state. When EN = 0, the counter holds its current state (Q(next) = Q for all bits). When EN = 1, the next state follows the selected sequence.

Table 4: State transition table of the special counter

Present State	M	EN	Next State
000	**0**	**1**	**001**
000	**1**	**1**	**010**
001	**X**	**1**	**100**

100	X	1	111
111	0	1	001
111	1	1	010
010	X	1	110
110	X	1	101
101	0	1	001
101	1	1	010

The state 011 is not used in the required counting sequences and is treated as an unused (don't-care) state during logic simplification.

3.6 JK Excitation Table for the Counter

The JK excitation table is used to determine the required J and K inputs for each flip-flop based on the transition from the present state to the next state.

For a single JK flip-flop:

Table 5:JK flip-flop excitation table

Present Q	Next Q(next)	J	K
0	0	0	X
0	1	1	X
1	0	X	1
1	1	X	0

Using the state transitions of the counter, the required excitation values for Q2, Q1, and Q0 are obtained as shown below.

Table 6:JK excitation table for the special counter

Present State	M	Next State	J2	K2	J1	K1	J0	K0
000	0	001	0	X	0	X	1	X
000	1	010	0	X	1	X	0	X
001	X	100	1	X	0	X	X	1
100	X	111	X	0	1	X	1	X
111	0	001	X	1	X	1	X	0
111	1	010	X	1	X	0	X	1
010	X	110	1	X	X	0	0	X
110	X	101	X	0	X	1	1	X
101	0	001	X	1	0	X	X	0
101	1	010	X	1	1	X	X	1

3.7 Karnaugh Maps

The excitation values obtained in the previous section were simplified using Karnaugh maps. The unused state 011, together with the JK excitation don't-care values, is treated as X to simplify the Boolean expressions. The Karnaugh maps are constructed for EN=1. The enable signal is added later as a common gating term in the final equations.

3.7.1 Karnaugh Maps for JK Excitation Inputs

Table 9: Karnaugh map for J2

Q2Q1 \ MQ0	00	01	11	10
00	0	1	1	0
01	1	X	X	0
11	X	X	X	X
10	X	X	X	X

The simplified expression is: J2 = Q1 + Q0

Table 10: Karnaugh map for K2.

Q2Q1 \ MQ0	00	01	11	10
00	X	X	X	X
01	X	X	X	X
11	0	1	1	0
10	0	1	1	0

The simplified expression is: K2 = Q0

Table 11:Karnaugh map for J1

Q2Q1 \ MQ0	00	01	11	10
00	0	0	0	1
01	X	X	X	X
11	X	X	X	X
10	1	0	1	1

The simplified expression is:
J1 = M·Q2 + M·Q0' + Q2·Q0'

Table 12: Karnaugh map for K1

Q2Q1 \ MQ0	00	01	11	10
00	X	X	X	X
01	0	X	X	0
11	1	1	0	1
10	X	X	X	X

The simplified expression is: K1 = Q0·M' + Q2·Q0'

Table 13: Karnaugh map for J0

Q2Q1 \ MQ0	00	01	11	10
00	1	X	X	0
01	0	X	X	0
11	1	X	X	1
10	1	X	X	1

The simplified expression is: J0 = Q2 + M'·Q1'

Table 14: Karnaugh map for K0

Q2Q1 \ MQ0	00	01	11	10
00	X	1	1	X
01	X	X	X	X
11	X	0	1	X
10	X	0	1	X

The simplified expression is: K0 = M + Q2'

Figure 3.2: K-maps

3.8 Simplified Counter Equations

After obtaining the simplified expressions from the Karnaugh maps, the synchronous enable signal is added as a common gating term.

The final equations used in the design are:

- J2 = EN · (Q1 + Q0)
- K2 = EN · Q0
- J1 = EN · (M·Q2 + M·Q0' + Q2·Q0')
- K1 = EN · (Q0·M' + Q2·Q0')
- J0 = EN · (Q2 + M'·Q1')
- K0 = EN · (M + Q2')

When EN = 0, all J and K inputs become zero, causing the three JK flip-flops to hold their current state.

3.9 Overall Structural Architecture

The overall design is organized into separate modules to keep the circuit clear and modular. The mode_logic block controls when the external selection input (S) is stored. The stored value (M) is then used by the counter_logic block to generate the required JK excitation signals for the three counter flip-flops.

The three positive-edge-triggered JK flip-flops store the counter state bits (Q_2), (Q_1), and (Q_0), while the negative-edge-triggered JK flip-flop stores the sequence selection bit (M). The counter outputs are also fed back to both the counter logic and the mode logic.

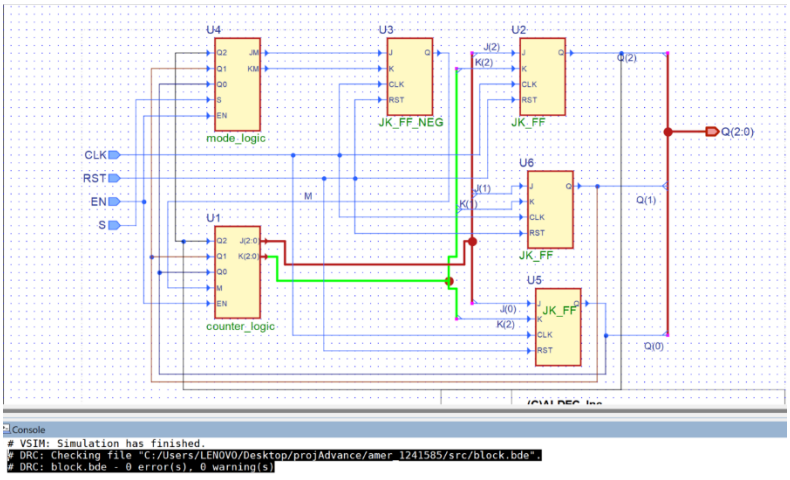


Figure 3.4:Overall structural block diagram of the special counter, as implemented in Active-HDL

The diagram shows the connection between the mode storage logic, the counter excitation logic, the JK flip-flops, and the external control signals (S), (EN), (CLK), and (RST).

3.10 Verilog Implementation

The design was implemented in Verilog using separate modules for the flip-flops, combinational excitation logic, mode storage logic, and the top-level structural connection. This modular organization makes the design easier to understand, test, and modify.

3.10.1 JK_FF Module

The JK_FF module implements a positive-edge-triggered JK flip-flop with an asynchronous reset. Three instances of this module are used to store the counter state bits (Q_2), (Q_1), and (Q_0).The asynchronous reset forces the output to zero immediately, while the normal JK operation occurs at the positive edge of the clock.

```
1  //JK Flip Flop
2  module JK_FF (J, K, CLK, RST, Q);
3
4      input J, K, CLK, RST;
5      output Q;
6      reg Q;
7
8      always @(posedge CLK or posedge RST)
9      begin
10         if (RST)
11             Q <= 1'b0;
12         else
13             case ({J,K})
14                 2'b00: Q <= Q;
15                 2'b01: Q <= 1'b0;
16                 2'b10: Q <= 1'b1;
17                 2'b11: Q <= ~Q;
18             endcase
19     end
20
21 endmodule
```

Figure 3.5: JK_FF Module code

3.10.2 JK_FF_NEG Module

The JK_FF_NEG module has the same JK operation, but it is triggered on the negative edge of the clock. It is used to store the sequence selection bit (M).

```
23 // Negative-edge JK flip-flop used to store the selected sequence
24 module JK_FF_NEG (J, K, CLK, RST, Q);
25
26     input J, K, CLK, RST;
27     output Q;
28     reg Q;
29
30     always @(negedge CLK or posedge RST)
31     begin
32         if (RST)
33             Q <= 1'b0;
34         else
35             case ({J,K})
36                 2'b00: Q <= Q;
37                 2'b01: Q <= 1'b0;
38                 2'b10: Q <= 1'b1;
39                 2'b11: Q <= ~Q;
40             endcase
41     end
42
43 endmodule
```

Figure 3.6 : JK_FF _NEG Module code

3.10.3 counter_logic Module

The counter_logic module implements the simplified Boolean equations obtained from the Karnaugh maps. Its inputs are the current state bits, the stored selection bit (M), and the synchronous enable signal.It generates the six excitation signals required by the three counter JK flip-flops:

```
// JK excitation logic for the counter
module counter_logic (Q2, Q1, Q0, M, EN, J, K);

    input Q2, Q1, Q0, M, EN;
    output [2:0] J, K;

    assign J[2] = EN & (Q1 | Q0);
    assign K[2] = EN & Q0;

    assign J[1] = EN & ((M  & Q2)  |(M  & ~Q0) |(Q2 & ~Q0));

    assign K[1] = EN & ((Q0 & ~M) |(Q2 & ~Q0));

    assign J[0] = EN & (Q2 |(~M & ~Q1));

    assign K[0] = EN & (M |~Q2);

endmodule
```

Figure 3.7: counter_logic Module code

3.10.4 mode_logic Module

The mode_logic module determines when the external selection input (S) can be stored in (M). The capture condition is active at the initial state and at the end of either counting cycle.

```
// Sequence selection storage logic
module mode_logic (Q2, Q1, Q0, S, EN, JM, KM);

    input Q2, Q1, Q0, S, EN;
    output JM, KM;

    wire C;

    assign C = EN & ((Q2 & Q0) |(~Q2 & ~Q1 & ~Q0));

    assign JM = C & S;
    assign KM = C & ~S;

endmodule
```

Figure 3.8: mode_logic Module code

If the capture condition is inactive, both (J_M) and (K_M) are zero, so the stored mode remains unchanged.

3.10.5 special_counter Top Module

The special_counter module is the top-level structural module. It connects the mode logic, counter logic, the three positive-edge JK flip-flops, and the negative-edge mode flip-flop.

The counter state is formed from the three flip-flop outputs:

```
// Top-level structural module of the special counter
module special_counter (CLK, RST, EN, S, Q);

    input CLK, RST, EN, S;
    output [2:0] Q;

    wire Q2, Q1, Q0;
    wire M;

    wire [2:0] J, K;
    wire JM, KM;

    counter_logic L1 (Q2, Q1, Q0,M, EN,J, K);

    mode_logic L2 (Q2, Q1, Q0,S, EN,JM, KM);

    JK_FF FF2 (J[2], K[2],CLK, RST,Q2);

    JK_FF FF1 (J[1], K[1],CLK, RST,Q1);

    JK_FF FF0 (J[0], K[0],CLK, RST,Q0);

    JK_FF_NEG FFM (JM, KM,CLK, RST,M);

    assign Q = {Q2, Q1, Q0};

endmodule
```

Figure 3.9:special_counter Top Module code

4 Verification and Simulation Results

The special counter was verified using a self-checking Verilog testbench. The testbench applies different values of RST, EN, and S, then compares the actual output Q with the expected counter state. If the output is incorrect, an error message is printed and the error counter is incremented.

4.1 Testbench Methodology

A clock signal is generated continuously using: always #5 CLK = ~CLK;

A task named check_Q is used to check the output after each positive clock edge.

```
115     task check_Q;
116         input [2:0] expected;
117         begin
118             @(posedge CLK);
119             #1;
120 
121             if (Q !== expected)
122             begin
123                 $display("ERROR at time %0t: expected Q = %b, actual Q = %b",$time, expected, Q);
124                 errors = errors + 1;
125             end
126             else
127             begin
128                 $display("PASS at time %0t: Q = %b",$time, Q);
129             end
130         end
131     endtask
```

Figure 4.1:Verilog self-checking testbench used to verify the special counter

4.2 Asynchronous Reset Test

At the beginning of the simulation, RST is asserted and the output is checked immediately. The expected initial state is: Q = 000

The testbench also applies the asynchronous reset again during normal operation to confirm that the counter returns directly to 000.

```
133     initial
134     begin
135 
136         CLK    = 0;
137         RST    = 1;
138         EN     = 0;
139         S      = 0;
140         errors = 0;
141 
142         #2;
144         if (Q !== 3'b000)
145         begin
146             $display("ERROR: Asynchronous reset failed");
147             errors = errors + 1;
148         end
149         else
150             $display("PASS: Reset -> Q = 000");
```

Figure 4.3: Asynchronous reset during operation test

```
210         RST = 1;
211 
212         #1;
213 
214         if (Q !== 3'b000)
215         begin
216             $display("ERROR: Asynchronous reset during operation failed");
217             errors = errors + 1;
218         end
219         else
220             $display("PASS: Asynchronous reset during operation -> Q = 000");
```

Figure 4.2: Initial asynchronous reset test

4.3 Sequence A Test

After reset, the counter is enabled with: EN = 1 & S=0.

The expected sequence is: 001 → 100 → 111

which corresponds to: 1 → 4 → 7

The testbench repeats this sequence to verify continuous correct operation.

4.4 Enable Hold Test

After reaching state `111`, the enable input is set to:

EN = 0

The testbench checks that the counter remains at:

Q = 111

for multiple clock cycles. This verifies that the synchronous enable correctly holds the current state.

4.5 Selection Change While Disabled

While `EN=0`, the selection input `S` is changed from 0 to 1. The counter remains at `111`, showing that changing the selection input does not change the counter state while counting is disabled.

After `EN` returns to 1, the new selection is used for the next sequence.

4.6 Sequence B Test

With the stored selection set for Sequence B, the expected sequence is:

010 → 110 → 101

which corresponds to: 2 → 6 → 5

The testbench repeats this sequence and verifies every state.

4.7 Switching from Sequence B to Sequence A

The selection input is changed back to `S=0`. After the current Sequence B cycle is completed, the counter starts Sequence A again:

001 → 100 → 111

This verifies correct switching between the two required sequences.

4.8 Selection Change During an Active Cycle

The testbench changes `S` during Sequence A after the counter has already entered the cycle.

The counter continues the current sequence:

001 → 100 → 111

and only starts the newly selected Sequence B after the current cycle is completed:

010 → 110 → 101

This confirms that the selection input does not interrupt an active sequence cycle

4.9 Simulation Waveform

The simulation waveform shows the behavior of CLK, RST, EN, S, and the three-bit output Q.

The waveform confirms the following behavior:

- Sequence A follows 1 → 4 → 7.
- Sequence B follows 2 → 6 → 5.
- The output holds its state when EN=0.
- A change in S does not interrupt the current sequence cycle.
- The asynchronous reset returns the output to 000.

The console output shows the result of each self-checking comparison. Each expected state produced a PASS message, and the asynchronous reset tests also passed.

At the end of the simulation, the testbench displayed:

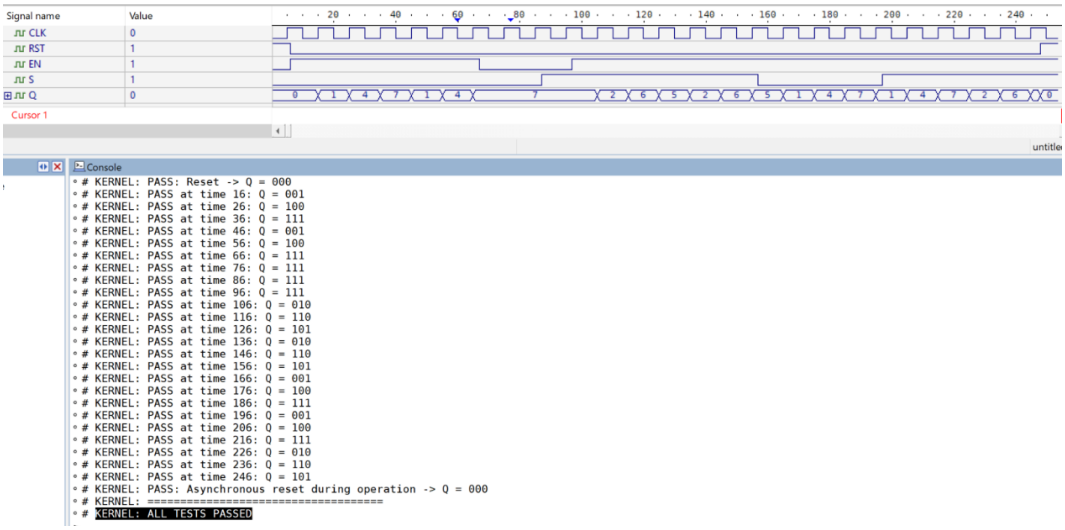


Figure 4.4: Console output of the self-checking testbench showing that all test cases passed successfully.

5 Discussion

The simulation results confirm that the counter satisfies the required behavior. Both sequences were generated correctly: 1 → 4 → 7 and 2 → 6 → 5. The asynchronous reset correctly returns the counter to 000, EN = 0 holds the current state, and changing S during an active cycle does not interrupt the current sequence — the new selection is applied to the following cycle.

5.1 Design Decisions

The main design choices — storing the selection in a separate bit M, capturing it on the negative clock edge, and gating the excitation equations with EN — together ensure that the counter's behavior stays predictable: the active sequence never changes mid-cycle, the new selection is always ready before it is needed, and disabling the counter never produces an unintended state

change. These choices add a small amount of extra logic but avoid more complex control structures such as multiplexers or additional state bits.

5.2 Advantages and Limitations

The modular design separates the counter logic, mode logic, and flip-flops, making the circuit easier to understand and test. The stored mode also prevents sequence changes from disturbing an active cycle.

A limitation is that the design uses both positive and negative clock edges, which adds timing considerations. In addition, S should be stable when it is sampled, and the unused state 011 is treated as a don't-care condition.

5.3 Suggested Improvements

One possible improvement is to redesign the sequence-storage mechanism so that all flip-flops operate on the same clock edge. This could simplify timing analysis, although additional control logic may be required.

Another possible improvement is to define an explicit recovery transition from the unused state 011 to the initial state. This would make the counter more robust if an unexpected state occurs.

The verification could also be extended with additional test cases, such as repeated random changes of S and EN, to test the design under a wider range of input combinations.

6 Conclusion

This project presented the design, implementation, and verification of a special digital counter using JK flip-flops and combinational logic. The counter was designed to generate two different sequences: $1 \rightarrow 4 \rightarrow 7$

and $2 \rightarrow 6 \rightarrow 5$

A stored mode bit was used to keep the selected sequence unchanged during an active counting cycle. The design also included an asynchronous reset and a synchronous enable input.

The circuit was implemented in Verilog using separate modules for the JK flip-flops, counter excitation logic, mode-storage logic, and top-level structural connections. The required excitation equations were obtained from the state transitions and simplified using Karnaugh maps.

The design was verified using a self-checking testbench in Active-HDL. The simulation confirmed correct operation of both counting sequences, reset behavior, enable hold and resume, and sequence switching. All implemented test cases passed successfully.

Overall, the project demonstrates the complete design process of a sequential circuit, starting from state definition and JK excitation analysis and ending with structural Verilog implementation and simulation verification.